

Manual Técnico

Sistema Hospitalario Inteligente — Proyecto 2, Grupo 4

1. Objetivo y alcance

Sistema de gestión hospitalaria con 10 módulos funcionales (Pacientes, Consultas, Hospitalización, Laboratorio, Farmacia, Radiología, Facturación, Emergencias, Agenda médica, Inventario), más módulos de soporte (Usuarios, Auditoría, Dashboard, Perfil, Notificaciones). Construido cumpliendo la restricción de **no usar frameworks**: PHP puro, MySQL, HTML/CSS/JavaScript/jQuery/AJAX.

2. Stack tecnológico

Capa	Tecnología
Backend	PHP 8.2 (sin framework), arquitectura por capas hecha a mano
Base de datos	MySQL/MariaDB (InnoDB, utf8mb4)
Frontend	HTML5, CSS3 (variables CSS, sin preprocesadores), JavaScript + jQuery
Comunicación	AJAX (JSON) — ningún CRUD recarga la página
Reportes	TCPDF (PDF), PhpSpreadsheet (Excel) — únicas dependencias vía Composer
Gráficas	Chart.js
Servidor de pruebas	Servidor embebido de PHP / Apache (XAMPP)

3. Arquitectura

Router → Controller → Model → (Vista PHP + JS/AJAX)

- `public/index.php`: front controller. Todas las peticiones a rutas de API pasan por aquí; las páginas `.php` y assets estáticos se sirven directamente.
- `src/core/Router.php`: mapea método HTTP + patrón de ruta (`{id}`) a un método de controlador.
- `src/core/Model.php`: clase base con CRUD genérico sobre PDO (paginación, búsqueda `LIKE`, soft delete opcional).

- `src/core/CrudController.php`: clase base que expone `index/show/store/update/destroy` para modelos simples, aplicando automáticamente autenticación, autorización por rol, CSRF y auditoría.
- Controladores con lógica de negocio más compleja (transacciones multi-tabla, validaciones cruzadas) **no** extienden `CrudController` y escriben SQL a medida (ej. `HospitalizationController`, `PharmacyController`, `LaboratoryController`).
- `src/views/partials/header.php` / `footer.php`: layout compartido (sidebar, topbar, notificaciones) incluido por las 15 páginas de `public/`.

Autoloading

`public/index.php` registra un `spl_autoload_register` que busca la clase solicitada en `src/{core,middleware,controllers,models,reports}/{Clase}.php` — no se requiere `require` manual al agregar controladores/modelos nuevos.

4. Base de datos

- **67 tablas** agrupadas en 14 módulos (ver `database/schema.sql` para el detalle completo, con comentarios de sección).
- Relaciones 1:1 (`medicos`↔`personal`), 1:N (`pacientes`→`citas`) y N:M (`medico_especialidades`, `paciente_alergias`, `rol_permisos`) con claves foráneas explícitas.
- **5 vistas** (`vw_ocupacion_camas`, `vw_citas_hoy`, `vw_stock_bajo`, `vw_examenes_pendientes`, `vw_facturacion_dia`) — consumidas por el Dashboard.
- **4 triggers**: `trg_ingreso_after_insert`/`trg_alta_after_insert`/`trg_traslado_after_insert` (mantienen el estado de `camas` sincronizado automáticamente), `trg_dispensacion_after_insert` (descuenta stock del lote más próximo a vencer).
- **2 procedimientos almacenados**: `sp_dispensar_receta` (transaccional, valida stock y lanza `SIGNAL` de error de negocio si es insuficiente), `sp_registrar_pago_factura` (registra el pago y recalcula el estado de la factura).
- **2 funciones**: `fn_edad_paciente`, `fn_stock_medicamento`.
- Scripts de importación, en orden, documentados en `README.md`.

5. Seguridad

Requisito	Implementación
Hash de contraseñas	<code>password_hash()</code> / <code>password_verify()</code> (bcrypt)
Sesiones seguras	Sesión nativa de PHP, cookie <code>HttpOnly</code> + <code>SameSite=Lax</code>
CSRF	Token por sesión, validado en cada <code>POST/PUT/DELETE</code> vía header <code>X-CSRF-Token</code> (<code>src/core/Csrf.php</code>)

Requisito	Implementación
XSS	<code>htmlspecialchars()</code> en toda salida PHP a HTML; en JS, inserción de contenido dinámico vía <code>.text()</code> de jQuery (nunca <code>.html()</code> con datos de usuario sin escapar)
SQL Injection	100% de las consultas usan <code>PDO::prepare()</code> con parámetros — cero concatenación de SQL con datos de entrada
Bloqueo por intentos fallidos	<code>intentos_login</code> cuenta fallos en ventana de 15 min; tras 5, <code>usuarios.bloqueado_hasta</code> se fija (comparación hecha en SQL, no en PHP, para evitar problemas de zona horaria)
Recuperación de contraseña	Token de un solo uso con expiración (<code>tokens_recuperacion_password</code>)
Auditoría	Cada <code>create/update/delete</code> inserta en <code>bitacora_auditoria</code> (tabla, registro, acción, usuario, diff JSON)
Control de acceso por rol	<code>RoleMiddleware</code> en cada endpoint backend + <code>src/core/Permissions.php</code> en el frontend (oculta módulos sin acceso y bloquea la URL directa)

6. Estructura de carpetas

```

database/ Scripts SQL (schema, views, triggers, procedures, seeds)
public/ Document root
*.php 15 páginas (una por módulo + login/perfil/auditoría)
assets/css/ style.css (sistema de diseño, variables CSS)
assets/js/ app.js (helpers globales), crud-module.js (CRUD genérico + modal)
assets/js/modules/ un archivo por página con su lógica específica
src/
config/ Conexión PDO y configuración de la app
core/ Router, Model, CrudController, Auth, Csrf, Icons, Permissions
middleware/ AuthMiddleware, RoleMiddleware
controllers/ ~35 controladores (uno o varios por módulo)
models/ ~40 modelos (uno por tabla principal)
reports/ PdfReport.php, ExcelReport.php (envoltorios de TCPDF/PhpSpreadsheet)
views/partials/ header.php, footer.php (layout compartido)
docs/ Este manual, el manual de usuario y el guion de video
PROGRESS.md Bitácora fase por fase de todo el desarrollo

```

7. Referencia de API (resumen)

Todas las rutas bajo `/api` implícito (servidas desde la raíz vía el router). Autenticación por cookie de sesión; escritura requiere header `X-CSRF-Token`.

Módulo	Rutas principales
Auth	<code>POST /auth/login</code> , <code>POST /auth/logout</code> , <code>GET /auth/me</code> , <code>PUT /auth/profile</code> , <code>POST /auth/password-reset/request</code> , <code>POST /auth/password-reset/confirm</code>
Pacientes	<code>GET/POST /patients</code> , <code>PUT/DELETE /patients/{id}</code> , <code>GET /patients/{id}/medical-record</code>
Personal	<code>GET/POST /staff/doctors</code> , <code>GET/POST /staff/nurses</code> , <code>.../unlock</code>
Agenda	<code>GET/POST /appointments</code> , <code>GET/POST /availability</code>
Hospitalización	<code>GET/POST /hospitalization/admissions</code> , <code>POST /hospitalization/discharges</code> , <code>POST /hospitalization/transfers</code>
Laboratorio	<code>GET/POST /lab/orders</code> , <code>/lab/samples</code> , <code>/lab/results</code>
Farmacia	<code>GET/POST /pharmacy/prescriptions</code> , <code>/pharmacy/prescription-items</code> , <code>POST /pharmacy/dispense</code>
Radiología	<code>GET/POST /radiology/orders</code> , <code>/radiology/studies</code> , <code>/radiology/reports</code>
Facturación	<code>GET/POST /billing/invoices</code> , <code>POST /billing/payments</code>
Emergencias	<code>GET/POST /emergency/cases</code> , <code>POST /emergency/attend</code>
Inventario	<code>GET /inventory/items</code> , <code>POST /inventory/movements</code>
Dashboard	<code>GET /dashboard/summary</code> , <code>/bed-occupancy</code> , <code>/low-stock</code> , <code>/emergency-by-priority</code> , etc.

Módulo	Rutas principales
Reportes	GET /reports/patients.pdf, /invoices.xlsx, /low-stock.pdf, /lab-orders.pdf, /radiology-orders.pdf, /admissions.pdf, /emergency-cases.xlsx
Admin	GET/POST/PUT/DELETE /admin/users, POST /admin/users/{id}/unlock, GET /admin/roles, GET /admin/audit
Notificaciones	GET /notifications, PUT /notifications/{id}/read, POST /notifications/read-all

Todos los listados (GET de colección) aceptan `?q=` (búsqueda) y, donde aplica, `?page=&pageSize=`.

8. Despliegue

Ver [README.md](#) — resumen: `composer install`, crear BD, importar los 9 scripts SQL en orden, levantar con `php -S localhost:8000 -t public public/index.php` o Apache con Virtual Host apuntando a `public/`.

9. Limitaciones conocidas

- El envío de correo de recuperación de contraseña es simulado (el token se devuelve en la respuesta, no se envía por SMTP).
- La paginación es server-side solo en Pacientes/Personal/Agenda; en los demás módulos el filtro de búsqueda es client-side sobre el conjunto ya cargado.
- No hay suite de pruebas automatizadas (unitarias/integración).
- No se ha medido formalmente el tiempo de respuesta ni realizado auditoría de accesibilidad (lectores de pantalla).

10. Historial de desarrollo

[PROGRESS.md](#) documenta, fase por fase, cada decisión, cada bug real encontrado durante el desarrollo y su causa raíz — útil como referencia de por qué el código está estructurado como está.